

Reliability and Recovery of Amazon SQS Messaging under Injected Consumer and Downstream Failures

Anjaneya Reddy Gurram (Anji) | 24288853 | MSc Cloud Computing, National College of Ireland

Abstract

Amazon SQS configuration guidance comes mainly from steady-state throughput studies where consumers and downstream services do not fail. Kyrychenko et al. (2025) optimise batch size and visibility timeout under correct operation, leaving poorly characterised the failure regime that visibility timeout, maxReceiveCount, and dead-letter queues serve. This study asks how SQS configuration affects reliability and recovery under injected consumer and downstream failures. A two-arm artefact (synchronous API versus SQS-decoupled) uses AWS SAM/Terraform with fault injection; localsim backs confirmatory tests and a live AWS lite round (eu-west-1; destroy-after) supplies directional checks. On localsim cells (stats_H1_H2_H3.json; 350 runs), queue-arm loss was 0.0 across VT levels, so $VT \rightarrow \text{loss}$ is not estimable ($p_{\text{adj}}=1$); $MRC \rightarrow \text{recovery}$ fails to reject after Holm ($H=2.43$, $p_{\text{adj}}=1.0$), while exploratory DLQ is strong (~ 0.294 at $MRC\ 1$ versus ~ 0 at $MRC \geq 5$). Live lite ($n=1$) shows loss 0.0; $MRC=1$ yields DLQ 0.23 and success 0.805 versus $MRC=5$ with DLQ 0.0 and success 1.0. Steady-state optima should not transfer into failure regimes without measuring loss, duplicates, DLQ, and recovery. Confirmatory results remain localsim-backed; live AWS is lite; FIFO/broker comparisons out of scope.

Introduction

Cloud-native applications increasingly communicate through asynchronous message queues so that producers and consumers scale and fail independently. Amazon SQS is a widely used managed queue with at-least-once delivery, configurable visibility timeout, redrive to a dead-letter queue via maxReceiveCount, and tight integration with AWS Lambda. Prior SQS configuration work, notably Kyrychenko et al. (2025), characterises throughput and latency under steady load and recommends large batches with long visibility timeouts for that regime. Real deployments, however,

are not steady-state: consumers crash, handlers throw, and datastores reject or time out. In those regimes, visibility timeout governs how soon a message becomes visible again after an incomplete receive, and maxReceiveCount governs how many failed receives precede DLQ capture. Mis-tuning can amplify redelivery churn, dead-letter poison messages too early, or delay recovery of backlog. Academic and vendor guidance remain largely qualitative heuristics, and fault-injection studies rarely target queue-consumer configuration at message-level dependent variables. Practitioners operating Lambda-backed SQS pipelines and researchers extending resilience measurement into managed queues therefore need controlled evidence in the failure regime rather than throughput optima alone.

How does Amazon SQS configuration affect message reliability and recovery under injected consumer and downstream failures? The work sets out to quantify reliability under four fault types by measuring loss, duplicates, and DLQ capture across visibility timeout, maxReceiveCount, and batch size, with success defined by manifests and summary tables for the committed matrix and confirmatory tests that follow the pre-registered H1–H3 plan with Holm–Bonferroni correction. It also characterises recovery as time from fault cessation until backlog returns toward steady state and relates that timing to visibility timeout and MRC, reporting effect sizes where estimable. A further aim is to test whether throughput-favourable settings such as long visibility timeout or large batch remain favourable under fault, with an explicit reject or fail-to-reject decision on the committed hypotheses. Finally, the study delivers a reproducible artefact—SAM/Terraform application, local simulator, experiment runner, and committed manifests—with documented localsim H1–H3 and live lite evidence paths and verified stack destroy after live rounds.

Methodologically, the design is controlled and two-armed: a synchronous order-processing control versus an SQS-to-Lambda treatment, deterministic in-process fault injection, a discrete-event localsim for confirmatory power without continuous AWS spend, and a lite live AWS key-cell campaign for directional external check. Analysis uses non-parametric tests with Holm correction, bootstrap intervals where applicable, and strict separation of confirmatory versus exploratory claims; cost dollar figures outside measured run ledgers are treated as estimator projections only. Scope is limited to SQS Standard queues, Lambda event-source consumers, and application-level faults rather than availability-zone or SQS service outages. The localsim abstracts network jitter and cold starts; live

AWS evidence is lite with $n=1$ cells and capped concurrency; FIFO, multi-region, and alternative brokers are excluded, so absolute live timings may differ from simulation. Working assumptions include warm-ish Lambda behaviour in simulation, idempotent destination writes where the artefact implements them, synthetic order workloads representative only of the committed payload schema, and destroy-after-round isolation that prevents cross-run resource leakage, recognising that shared-account ConcurrentExecutions constraints may still block larger live n .

The remainder of the report proceeds as follows. Chapter 2 reviews queue configuration, fault injection, and serverless resilience literature and states the gap. Chapter 3 details methodology, variables, hypotheses, and validity threats. Chapter 4 covers design and implementation of the artefact and simulator. Chapter 5 reports evaluation, including localsim confirmatory results and live lite evidence. Chapter 6 concludes with contributions, limitations, and future work.